# Software Requirements Specification (SRS) for USC Patient Information System (USC-PIS)

## I. Introduction

### Purpose & Scope

The University of San Carlos (USC) Clinic mandates a transition from a fragmented, paper-based operations model to a fully digital Health Information System (HIS). The USC-PIS is engineered to centralize and secure student health information by capturing comprehensive medical histories, treatment plans, medications, and dental records. The system scope encompasses the complete digital replacement of the Daily Treatment Records (DTR), consultation management, medical certificate issuance, health awareness campaigns, automated patient notifications, and rigorous feedback analytics.

### Design & Implementation Constraints

The "Modernizing the USC-DC Clinic's Patient Information System" project is a web-based system developed under the timeframe and academic constraints of the CpE Design course sequence. As a student-led initiative, the following constraints and boundaries apply to the current deployment:

- **Hardware & Hosting Constraints:** The system relies on Heroku's basic tier hosting and PostgreSQL, deliberately excluding costly enterprise integrations like Single Sign-On (SSO).
- **Inventory Management:** Medication stock tracking is not included.
- **Billing Integration:** Advanced insurance processing and financial management are excluded.
- **Appointment Scheduling:** Consultations are strictly handled via a manual walk-in workflow as per clinic request.
- **Patient Medication Tracking:** This functionality is deferred to future iterations.

To maintain a strict boundary on the current system deployment, the following features are deliberately excluded from the current thesis scope and deferred to future enterprise phases:

### Definitions

- **PII/PHI (Personally Identifiable Information / Protected Health Information):** Sensitive patient data protected by the database architecture.

- **pgcrypto:** A native PostgreSQL extension integrated into the system for applying industry-standard, column-level symmetric encryption to sensitive data.
- **RBAC (Role-Based Access Control):** A security paradigm implemented via Django REST Framework (DRF) permission classes to restrict access dynamically.
- **FDI Notation:** The standard numerical tooth identification system (11-48) enforced via regex constraints within the Dental Records module.

# II. Overall Description

## System Perspective

The USC-PIS is built upon a modern, three-tier web architecture. The backend is powered by Django 5.0.2 and Django REST Framework, while the presentation layer operates on React 18 using a Vite build system and Material-UI components. The data layer relies on a dual-database environment featuring PostgreSQL for production and SQLite for local development, enforcing vendor-aware logic to prevent compatibility errors during testing. External enterprise integrations include **Heroku** for hosting, **Cloudinary** for scalable media storage, and the **Gmail API** utilizing OAuth 2.0 to bypass "less secure app" restrictions for clinical communications.

## User Classes & Characteristics

The system strictly enforces user authorizations across hierarchical classes:

| Role | Access Level & Capabilities |
|---|---|
| **Admin** | Possesses full system access, including global RBAC adjustments, system health monitoring, email system administration, and report generation. |
| **Doctor / Dentist** | Endowed with full administrative and clinical access, only the Doctor role is uniquely authorized to approve medical certificates. Both can prescribe treatments and access all reporting/feedback analytics. |
| **Nurse** | Supervisory and data-entry capacity; authorized to cater to patients, record vital signs, input treatment plans, and draft medical certificates for doctor approval. |
| **Staff** | Manages clinic administration, patient intake, certificate issuance, report generation, and health campaign creation. All other clinical roles (Admin, Doctor, Dentist, and Nurse) possess the authorization to perform all tasks and access all functionalities available to this Staff role |

| | |
|---|---|
| **Student / Faculty (Patient)** | Restricted to their own data; permitted only to read personal medical/dental timelines, submit visit-linked feedback, and view public health campaigns. |

### Hardware Interfaces

- Client Devices: A standard desktop computer, laptop, or mobile device capable of rendering modern web applications.
- Network: A stable broadband internet connection is required for real-time data exchange with the cloud servers.

### Software Interfaces

- **User Interface (UI):** A web-based Graphical User Interface (GUI) built on React 18 and Material-UI, ensuring responsive, low-latency interactions across all devices.
- **Backend Framework:** Django 5.0.2 acts as the core application layer, providing secure RESTful API endpoints.
- **Database Management:** PostgreSQL handles relational data storage, utilizing the pgcrypto extension for data-at-rest encryption.
- **External APIs:** The system interfaces with the Gmail API (via OAuth 2.0) for automated communication, bypassing strict security blocks, and Cloudinary for scalable Content Delivery Network (CDN) file storage.

# III. Functional Requirements

- **Identity Management (*authentication*):** Secures access via the ***/api/auth/*** mount point. **USC domain enforcement** mandates that student registrations end in ***@usc.edu.ph***. A **SafeList registration** feature maps authorized clinical emails to roles, bypassing standard MFA logic to automatically assign professional access.
- **Clinical Records & Retooling (*patients*):** Data entry uses a unified record component (***MedicalRecord.jsx and Dental.jsx***). **Retooling Focus:** An advanced real-time filtering architecture processes queries via dynamic parameter filtering (***year_level and academic_year***). This logic calculates Semesters autonomously to sort students accurately.
- **Health Awareness (*health_info*):** Manages the lifecycle of clinical announcements across 13 defined categories (e.g., VACCINATION). Visibility is purely governed by ***start_date*** and ***end_date*** parameters; manual status toggling is deprecated.
- **Administrative Workflows (*medical_certificates*):** Digitalizes the USC Form ACA-HSD-04F workflow. Nurses draft the certificate, but Doctors hold exclusive authority (***IsDoctorUser***) to approve the final document.
- **Feedback & Notifications (*feedback, notifications*):** Feedback is linked uniquely to clinical visits via backend constraints, restricting users to one submission per visit. Creating a health record executes a signal that schedules a mandatory 24-hour follow-up alert.

- **Reports (*reports*):** Supports background processing logic (via Celery/Redis) to dynamically synthesize clinic reports, exporting data natively to PDF, Excel, and CSV without blocking HTTP threads.

# USE CASE DIAGRAM

## IV. Data & Security Requirements

- **Encryption-at-Rest:** Implements industry-level data encryption mapped to PostgreSQL's ***pgcrypto*** extension. Fields such as ***illness_enc*** and ***diagnosis_enc*** bypass plain-text memory and are converted strictly to ***BinaryField*** bytea hashes using backend signals upon database commits.
- **Access Control:** Guarded globally by RBAC middleware using DRF ***PermissionClasses***. Unauthenticated traffic is blocked by default. Files accessed under ***/api/files/patient-documents/*** are proxied strictly through authenticated backend streaming, blocking public CDN bypass attempts.

### System Accountability and Audit Logging

To ensure data integrity, operational transparency, and adherence to healthcare compliance standards, the USC-PIS implements a comprehensive tracking and logging architecture across its core modules.

- **Database Audit Trails:** The system enforces strict data accountability by embedding structural audit trails within the database schema. All critical entities, including patient profiles, medical records, and dental records, are automatically equipped with `created_at` and `updated_at` timestamp fields. This guarantees a precise, immutable history of when clinical data was first entered and subsequently modified.
- **System Activity and Notification Logging:** To monitor administrative oversight and system interactions, the USC-PIS maintains a persistent log of all dispatched email and in-app communications. Administrators have access to detailed "System Activity Logs" via the Email Administration dashboard, which provides timestamped records of backend email events and error diagnostics. Crucially, this creates an explicit audit trail that tracks delivery statuses and identifies exactly which administrator triggered specific manual actions or system overrides.

- **API Log Access and Compliance:** All user actions and email administration tasks are systematically logged to prevent unauthorized system manipulation and to ensure security. Furthermore, any manual email dispatches executed by the staff are explicitly documented for audit purposes. Authorized administrators can programmatically review these communication audit trails via the `/api/notifications/logs/` endpoint.

# V. Non-Functional Requirements & Future Roadmap

## Performance & Security Benchmarks

System performance benchmarking explicitly addressed and resolved prior inefficiencies:

| Requirement Area | Verified Benchmark | Audit Test ID |
|---|---|---|
| **Search Latency** | Patient searches execute in **127.81ms** (Target: <500ms). | PT-04 |
| **PDF Generation** | Medical Certificate PDFs render in **122.93ms**. | PT-01 |
| **Concurrency** | Processes 20 concurrent requests in **0.66 seconds** (0% failure). | PT-02 |
| **File Security** | **100% rejection rate** against executable scripts (.EXE, .JS). | UT-06 |

## Data Resilience

**Backup and Recovery:** The system is equipped with an automated data resilience module that allows authorized administrators to export the entire structured database, including patient identifiers, medical histories, and user accounts, into a single, highly compressed JSON file. This ensures business continuity and provides a rapid recovery point in the event of environment-level failures.

## Future Roadmap

While current features solve immediate inefficiencies, the system's scalable architecture supports future enhancements:

- **Mobile Progressive Web App (PWA):** Deploying a mobile application wrapper with offline capabilities and push notifications.
- **Telemedicine Platform:** Expanding healthcare delivery to include video consultations and remote monitoring.
- **AI/ML Integration:** Implementing machine learning models for predictive health analytics and diagnostic assistance.

# Comprehensive Training Plan (Deliverable D4): USC-PIS

## I. Program Overview

**Objectives:** The primary objective is to equip all users with the digital competency required to operate the fully digital USC-PIS, reducing patient record retrieval time to well under 1 second (capitalizing on the 127.81ms search benchmark). **Delivery Methods:** Training will be executed via a hybrid delivery approach, using hands-on demonstration modules supplemented by the comprehensive User Manuals.

## II. System Adoption Strategy (Addressing Bottlenecks)

Initial database audits revealed critical system underutilization (e.g., 3 medical records for 9 patients, 1 dental record, and 0 file uploads). To solve this adoption issue, training will incorporate:

- **Workflow Integration:** Practical demonstrations on incorporating the software directly into the clinic's daily operational workflows.
- **Incentives & Benefits:** Showcasing how digital record-keeping eliminates redundant paperwork and provides instant clinical insights.

## III. Module 1: Clinical Operations (For Doctors & Nurses)

- **Patient Intake:** Staff will learn to utilize the **Application Retooling Filter Bar** to segment patient datasets in real-time by **Role, Program, Year Level, and Registration Period**. Instruction will explicitly cover the Academic Year and Semester logic (e.g., filtering 1st Semester from August 1 to December 31) to route targeted student groups.

- **Record Management & Security:** Users will master the **Unified Record Component** for entering vitals and the mandatory "Concern / Reason for Visit" fields. A highly technical segment will focus on the **pgcrypto encryption architecture**, explaining how patient diagnoses and names are automatically hashed as secure ***BinaryField*** strings via backend signals.

# IV. Module 2: Specialized Clinical Workflows (For Dentists)

- **Dental Charting & Streamlined Entry:** Dentists will be instructed on charting tooth conditions using the **FDI notation (11-48)** numerical range, strictly enforced via regex. To accelerate high-volume clinic days, Dentists will master the **"Consultation Only" mode**, which makes full diagnoses optional, and utilize the explicit "Referral To" field.

# V. Module 3: Administrative Output (For Staff & Admins)

- **Certificate Issuance:** Staff will learn the workflow for generating the **ACA-HSD-04F Medical Certificate** directly from a health record. Training will emphasize that while Nurses may draft the certificate, the system mandates secure approval executed solely by the **Doctor** before the PDF is rendered.
- **Reporting:** Staff will practice generating **Visit Trends** and **Treatment Outcome Reports** using the Celery-powered asynchronous reporting module, achieving data outputs in under 1 second without blocking the interface.

# VI. Module 4: Student Engagement (For Students & Faculty)

- **Personal Health Access & Feedback:** Students will be trained on navigating their personalized **Student Medical Record (SMR)** via the ***/health-insights*** tab to view non-interactive timelines. They will also learn how to respond to automated post-visit emails by submitting evaluations and interacting with Health Campaigns.

# VII. Assessment & Evaluation

**Training Success & Adoption Metrics:** Rather than utilizing formal proficiency exams, the success of the training program and system adoption will be evaluated during the Pilot Deployment phase (targeting 70% of 1st-year CpE students and selected clinic staff). The evaluation will be based on two realistic pillars:

- **Direct User Feedback & Issue Reporting:** Success will be measured by qualitative feedback gathered from the clinic staff and pilot students regarding the system's intuitiveness, as well as tracking any reported bugs or workflow bottlenecks.
- **Real-World System Performance:** Evaluating whether trained staff can execute operational tasks efficiently under real clinic loads, ensuring that benchmarks, such as the sub-500ms patient search latency and rapid PDF generation, are maintained during daily operations.

# Appendix I: USC-PIS User Manuals

## Manual 1: Clinical & Administrative Staff Guide

**Target Audience:** Doctors, Nurses, Dentists, and Clinic Staff.

### 1. Secure Onboarding

1. Navigate to the registration portal and enter your pre-authorized clinic email address.
2. The system automatically cross-references the email against the backend ***SafeEmail*** table.
3. Upon validation, the system bypasses the 6-digit Multi-Factor Authentication (MFA) step and instantly assigns your professional clinical role.

**Technical Note: RBAC & Security:** Staff authorizations are globally protected via Django REST Framework ***PermissionClasses***. Access to patient databases is strictly gated to medical personnel, neutralizing unauthorized data exposure.

### 2. Advanced Patient Discovery

1. Navigate to the **Patients** module and use the global search bar to query a patient's Name, Email, or USC ID.
2. Click the **Filters** button to expand the Application Retooling Filter Bar.
3. Apply the **Academic Year** filter (e.g., 2025-2026), **Semester** logic (1st Sem: Aug-Dec, 2nd Sem: Jan-May), and **Year Level** to isolate specific student groups.

**Technical Note: Search Efficiency:** Database query optimizations verify that patient search results among 100+ records are returned with a latency of **127.81ms**, nearly four times faster than the benchmark limit.

## 3. The Unified Record Workflow

- **Medical Documentation:** Navigate to the **Medical Records** tab, input the mandatory "Concern / Reason for Visit", record vital signs, and click "Save".

  **Technical Note:** Upon saving, PostgreSQL's ***pgp_sym_encrypt*** function hashes sensitive patient data into a secure binary format invisibly in the background (< 1.0ms overhead).

- **Dental Documentation:** Navigate to the **Dental Records** tab and use the **FDI 11-48** numerical system to input tooth conditions. For rapid checks, use the streamlined "Consultation/Referral" mode.

## 4. Medical Certificate Management & Issuance

1. Navigate to **Medical Certificates** -> **Create New Certificate**.
2. Select the patient; the system will autonomously autofill their Name, Year Level, and Full Course Name.
3. Select the **USC Clinic Template ACA-HSD-04F** and enter the "Purpose/Requirement".
4. A **DOCTOR** must assess the draft, select Medical Fitness Status ("Fit"/"Not Fit"), and authorize approval. The dual-engine PDF system will instantly render the certificate for download.

## 5. System Maintenance & Data Backups

- **Generating a Backup:** To ensure the safety of clinical records, Administrative Staff can trigger a full system backup via the Database Monitor page.
- **Mechanism:** The system captures all active records (Medical history, dental charts, and user profiles) and packages them into a portable JSON format.
- **Storage:** It is recommended to download and store this compressed file in a secure university-approved location monthly to ensure a permanent offline copy of the clinic's digital database.

## 6. Email Administration & Notification Templates

The system features a massive automated communication engine requiring administrative oversight.

| Management Tab | Description & Functionality |
|---|---|

| | |
|---|---|
| **Routing & Status** | Monitor the 7 core diagnostics (Database, Email, Backups, Security, Performance, Storage, Cache). Use the **Global Master Switch** to instantly disable all automated emails during emergencies. |
| **Email Templates** | Manage content lifecycle. Edit subjects and bodies using dynamic placeholders (e.g., ***{{user_name}}***, ***{{appointment_date}}***). |
| **Sent Notifications** | View a real-time history of all communications and track status (SENT, READ, FAILED). |
| **Staff Access** | Instantly enable or disable In-App Notifications and Email Alerts on a per-staff member basis. |

- **Customizing Notification Templates:** Access the Django Admin portal (***/admin***) and navigate to ***NotificationTemplates***. Utilize placeholders to personalize the messages, and test the template logic via the API before deploying.

### 6. Patient Health Insights & Enterprise Reporting

- **Health Insights (*/health-insights*):** Review unified medical and dental timelines. Document attachments listed within the unified timeline are rendered non-interactive to enforce strict data privacy.
- **Enterprise Reporting (*/reports*):** Access available analytical templates (e.g., Visit Trends, Feedback Analysis).

**Technical Note:** Heavy data aggregations are processed asynchronously via Celery and Redis. PDF generation utilizes a Dual-Engine strategy (***xhtml2pdf*** with a ***ReportLab*** fallback) and exports are programmatically capped at the first 200 items per section to maintain document stability. Excel/CSV exports offer unlimited deep data analysis.

# Manual 2: Student & Faculty Portal Guide

**Target Audience:** Patients (Students and Faculty).

### 1. Identity & MFA

1. Navigate to the registration portal and enter an official university email address.
2. The system enforces strict domain validation; the email must end in ***@usc.edu.ph***.
3. Check the university inbox for the automated **6-digit verification code** dispatched via the Gmail API.
4. Input the code into the portal before the 24-hour expiration threshold to activate your account.

## 2. The Student Medical Record (SMR)

1. Log into the portal and navigate to the **Patient Medical History** link in the sidebar (***/health-insights***).
2. The interface displays a Unified Timeline separated into Medical Records, Dental Records, and Health Insights.
3. Click the **Health Insights** tab to view personalized health analytics, monthly visit frequency patterns, and visual trend indicators.

**Technical Note: Data Privacy:** The Student Medical Record dashboard is governed by strict access controls limiting visibility exclusively to the record owner and authorized medical staff. All clinical documents are proxied securely through an authenticated backend stream.

## 3. Health Awareness

1. Navigate to the **Campaigns** tab (***/campaigns***) in the navigation sidebar.
2. Browse active health awareness campaigns, which are categorically organized.
3. Select a campaign to open the **Universal Document Viewer**.
4. Interact with high-quality image PubMats via zooming, or execute direct, secure downloads of attached health guidelines.

## 4. The Feedback Loop

1. Exactly 24 hours following the conclusion of a Medical or Dental consultation, the system automatically dispatches a Feedback Request reminder to your university email.
2. Open the email and click the embedded rating links, or log directly into the portal and navigate to the **Feedback** tab.
3. Select the specific clinical visit to evaluate.
4. Input a 1-5 star "Overall Experience" rating, answer Yes/No to the "Staff Courtesy" metric, and provide qualitative comments.

**Technical Note: Feedback Constraints:** To ensure data integrity, the system architecture implements a ***UniqueConstraint*** on the backend. This restricts users to exactly one feedback submission per clinical visit, preventing skewed satisfaction analytics.

# Appendix G: Technical Audit and Validation Summary

## I. Software Quality Assurance (SQA) Audit Summary

The system's operational efficiency was rigorously evaluated under the quantitative parameters defined in the performance test suite (PT-01 to PT-04). The benchmarks confirm that the architectural optimizations successfully eliminated legacy latency issues. Specifically, search query optimizations yield a latency ranging strictly between **123.16ms and 127.81ms**, nearly four times faster than the 500ms threshold. Furthermore, the dual-engine PDF module generates official Medical Certificates in **122.93ms to 124.53ms**, safely below the 1000ms limit.

To guarantee reliability during peak registration, concurrency was audited via test PT-02. The architecture successfully handled **20 simultaneous HTTP requests in 0.66 seconds** with a 0% data drop rate. Additionally, a cybersecurity audit (UT-06) confirmed a **100% rejection rate** against high-risk extensions (.EXE, .JS, and .PY), neutralizing potential system vulnerabilities.

## II. Requirement Traceability Matrix (RTM)

The following matrix maps the core functional and security requirements derived from the SRS directly to their verified implementation and automated test outcomes.

| Requirement Category | Core Requirement Description | Verified Implementation | Audit Test ID | Status |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| **Data Security** | Column-Level Database Encryption | PostgreSQL ***pgcrypto*** binary storage | UT-01 | **PASS** |
| **Application Retooling** | Academic Year & Semester Filtering | Dynamic parameter filtering via ***PatientViewSet*** | IT-04 | **PASS** |
| **Clinical Logic** | FDI Notation Enforcement (11-48) | Strict regex validation within Dental records | UT-04 | **PASS** |
| **System Security** | Secure File Upload Integrity | Interception and rejection of executable scripts | UT-06 | **PASS** |
| **Clinical Workflows** | End-to-End Medical Certification | Issuance pipeline: Nurse draft to Doctor approval | IT-01 | **PASS** |

## III. Technical Dossier & API Authorization Map

The system architecture mandates strict Role-Based Access Control (RBAC) using Django REST Framework (DRF) ***PermissionClasses*** to shield clinical data.

| API Endpoint | HTTP Method | DRF Permission Class | Authorized Roles |
|---|---|---|---|
| /api/auth/profile/me/ | GET, PATCH | IsAuthenticated | All Roles (Own Data) |
| /api/patients/ | GET, POST | IsStaffUser | ADMIN, STAFF, DOCTOR, NURSE, DENTIST |
| /api/certificates/{id}/assess/ | POST | IsDoctorUser | DOCTOR |
| /api/patients/records/ | POST, PATCH | MedicalRecordPermission | DOCTOR, NURSE, DENTIST |

## IV. Data Privacy & Encryption Workflow

To prevent unauthorized access to sensitive clinical attachments, the system enforces a **Secure Download Proxy** at ***/api/files/patient-documents/{id}/download/.*** Documents are streamed securely through the backend server utilizing full Django authentication, preventing any bypass of the CDN.

The encryption workflow integrates industry-standard protection directly at the database level using native PostgreSQL ***pgcrypto*** functions. Upon a database commit involving Protected Health Information (PHI), a Django ***post_save*** signal intercepts the plaintext payload and executes the ***pgp_sym_encrypt*** function. This converts sensitive data, such as clinical diagnoses, strictly into secure ***BinaryField*** bytea hashes before permanent storage on the disk.

## V. Data Resilience & Disaster Recovery Audit

**Data Resilience Module:** The USC-PIS architecture is equipped with a custom-engineered JSON Backup Engine managed by the ***utils.services.BackupService*** module. To ensure business continuity and protect sensitive clinical records, this engine performs a structured export of the entire database, including patient identifiers, medical histories, dental charts, and user profiles, packaging the relational data into a portable, highly compressed JSON payload.

**Environment Portability & Restoration:** This JSON-based backup architecture was explicitly designed to bypass vendor-specific SQL constraints, facilitating seamless, high-fidelity data migration between the Production and Development environments. Authorized administrative staff can manually trigger full system backups directly via the Database Monitor page to guarantee a permanent, offline recovery point in the event of environment-level failures.

**Integrity Verification & Automated Alerts:** To guarantee data fidelity upon restoration, the backup engine integrates a complete ***create/verify/restore*** lifecycle. It utilizes post-export verification logic to ensure that sensitive ***pgcrypto*** binary hashes remain intact and uncorrupted during the compression process.

# Appendix I: Software Design Document (SDD)

## 1. System Architecture Design

### Three-Tier Architecture

The USC-DC Patient Information System (USC-PIS) is engineered upon a modern, highly scalable three-tier web architecture.

- **Presentation Layer (Frontend):** Developed utilizing React 18 with a Vite build system and Material-UI components, providing a responsive, low-latency, and component-based Graphical User Interface (GUI) for clinic staff and patients.
- **Application Layer (Backend):** Powered by Python and Django 5.0.2, integrated with the Django REST Framework (DRF) 3.14.0. This layer encapsulates the core business logic, API routing, and Role-Based Access Control (RBAC) middleware.
- **Data Layer (Database):** Operates on a dual-database paradigm utilizing PostgreSQL 16.8 for production deployment and SQLite for local development. The system enforces vendor-aware logic to guarantee compatibility across environments.

### Enterprise Integrations

To support complex clinical operations, the architecture incorporates three primary enterprise integrations:

- **Heroku:** Serves as the primary cloud hosting platform, providing automated CI/CD pipelines, secure Dyno management, and managed PostgreSQL database hosting.
- **Cloudinary:** Operates as the system's scalable Content Delivery Network (CDN). It securely handles the persistent storage and rapid delivery of media, including Health Campaign PubMats and generated clinical PDF reports.
- **Gmail API (OAuth 2.0):** Replaces standard SMTP protocols to bypass "less secure app" restrictions. It utilizes OAuth 2.0 tokens to reliably dispatch high-priority clinical communications, including Multi-Factor Authentication (MFA) codes and automated feedback reminders.

## 2. Data Design & Dictionary

### Database Normalization

The database architecture is designed in alignment with Third Normal Form (3NF) principles to eliminate data redundancy and ensure relationship integrity. Patient demographic data is

logically decoupled from system authentication credentials (`authentication_user`), while all clinical visits and assessments are mapped securely via explicitly constrained Foreign Keys (FK) with cascading or null-setting safeguards.

## Data Dictionary

**Table 1: `authentication_user` (Core System User)**

| Field Name | Data Type | Description |
|---|---|---|
| `id` | Integer | Primary Key (PK). Unique identifier for the user. |
| `email` | String (varchar) | Unique Key (UK). Enforces `@usc.edu.ph` domain constraint. |
| `password` | String (varchar) | Securely hashed user password. |
| `role` | String (varchar) | RBAC identifier (ADMIN, DOCTOR, DENTIST, NURSE, STAFF, STUDENT). |
| `is_verified` | Boolean | Flag indicating successful MFA OTP verification. |
| `illness_enc` | BinaryField (bytea) | `pgcrypto` encrypted string of chronic illnesses. |

**Table 2: `patients_patient` (Clinical Profile)**

| Field Name | Data Type | Description |
|---|---|---|

| | | |
|---|---|---|
| `id` | Integer | Primary Key (PK). |
| `user_id` | Integer | Foreign Key (FK) mapping to `authentication_user.id`. |
| `usc_id` | String (varchar) | 8-digit official University ID number. |
| `first_name_enc` | BinaryField (bytea) | `pgcrypto` encrypted first name payload. |
| `last_name_enc` | BinaryField (bytea) | `pgcrypto` encrypted last name payload. |
| `created_at` | Datetime | Audit trail timestamp for record creation. |

**Table 3: `patients_medicalrecord` (Medical History)**

| Field Name | Data Type | Description |
|---|---|---|
| `id` | Integer | Primary Key (PK). |
| `patient_id` | Integer | Foreign Key (FK) mapping to `patients_patient.id`. |
| `created_by_id` | Integer | Foreign Key (FK) mapping to the attending medical staff's `user_id`. |

| | | |
|---|---|---|
| `concern` | String (varchar) | Mandatory chief complaint or reason for clinical visit. |
| `diagnosis_enc` | BinaryField (bytea) | `pgcrypto` encrypted clinical diagnosis payload. |
| `vital_signs` | JSON | Structured JSON array capturing BP, HR, Temp, etc. |

**Table 4: `patients_dentalrecord` (Dental History)**

| Field Name | Data Type | Description |
|---|---|---|
| `id` | Integer | Primary Key (PK). |
| `patient_id` | Integer | Foreign Key (FK) mapping to `patients_patient.id`. |
| `tooth_numbers` | String (varchar) | CSV string of affected teeth enforcing FDI Notation limits. |
| `procedure_performed` | String (varchar) | Defined choices constraint (e.g., CLEANING, EXTRACTION). |
| `tooth_chart` | JSON | Structured representation of the visual dental diagram. |

## Clinical Logic Constraints

- **FDI Tooth Notation Enforcement:** The dental module actively restricts charting to the standard FDI numerical tooth identification system. It enforces regex constraints limiting input strictly to the `11-48` range.
- **Encryption-at-Rest (`pgcrypto`):** Military-grade symmetric encryption (AES-256) is applied at the column level to safeguard Protected Health Information (PHI). Through Django `post_save` signals, the PostgreSQL `pgp_sym_encrypt` function is executed upon database commits, instantly converting plaintext fields (such as clinical diagnoses and patient names) into secure `BinaryField` (`bytea`) hashes.

# 3. Interface & Security Design

## API Map & RBAC Implementation

Access to backend endpoints is restricted via DRF `PermissionClasses` to enforce Role-Based Access Control (RBAC).

| RESTful API Endpoint | HTTP Methods | DRF Permission Class | Authorized Roles |
|---|---|---|---|
| `/api/auth/profile/me/` | GET, PATCH | `IsAuthenticated` | All Roles (Self-Data Only) |
| `/api/patients/` | GET, POST | `IsStaffUser` | ADMIN, STAFF, DOCTOR, NURSE, DENTIST |
| `/api/patients/records/` | POST, PATCH | `MedicalRecordPermission` | DOCTOR, NURSE, DENTIST |
| `/api/medical-certificates/` | GET, POST | `IsStaffOrMedicalPersonnel` | DOCTOR, NURSE |

| | | | |
|---|---|---|---|
| `/api/certificates/{id}/assess/` | POST | `IsDoctorUser` | DOCTOR (Exclusive Approval) |

### Secure Proxy for Patient Documents

To guarantee data isolation and prevent unauthorized file exposure, direct CDN URL access for clinical attachments is blocked. The system utilizes an Authenticated Download Proxy mapped at `/api/files/patient-documents/{id}/download/`. Upon request, the file stream is proxied directly through the Django backend, verifying the active user's session and authorization credentials before successfully dispensing the `.PDF` or `.JPG` payload.

### Audit Logging

The USC-PIS reinforces compliance standards via an immutable tracking architecture. Beyond standard `created_at` and `updated_at` database timestamps, the system implements custom middleware that automatically records the active User ID, the exact Timestamp, and the requested API Endpoint for every instance of clinical data access. This establishes a definitive audit trail to monitor user accountability and track system overrides by administrative staff.

## 4. Procedural & Notification Design

### MFA Flow & OTP Generation

To ensure stringent identity verification, the authentication module processes a 6-digit One-Time Password (OTP) sequence. Upon registration, the system strictly enforces the `@usc.edu.ph` domain constraint. Following validation, a secure OTP is generated, assigned a precise expiration timestamp (10 minutes), and transmitted directly to the user's inbox utilizing the Gmail API OAuth 2.0 integration.

### Asynchronous Tasks & Feedback Loop

To prevent HTTP bottlenecks during communication broadcasts, the architecture leverages a Celery and Redis pipeline for background task execution. When a `MedicalRecord` or `DentalRecord` is committed to the database, a `post_save` signal (`schedule_feedback_email_medical` or `schedule_feedback_email_dental`) is triggered. This executes an asynchronous task placing a 24-hour delayed job in the Celery queue. Exactly 24 hours post-visit, the automated feedback loop dispatches an evaluation request to the patient, encouraging continuous, data-driven service refinement.

# 5. Preliminary User Acceptance Testing (UAT) Summary

## Staff Evaluation

Preliminary User Acceptance Testing (UAT) was conducted utilizing feedback from a clinical sample of Clinic Staff (N=7, comprised of Nurses, Doctors, and Dentists). The quantitative and qualitative returns indicate strong efficiency improvements compared to the legacy manual operations. Staff explicitly noted that "generating data is easier" and that the workflow is "fast and easy". Early testing bottlenecks, such as the request to add a feature to sort students by academic year, were immediately verified and resolved during the Application Retooling phase. One Doctor highlighted rigorous expectations for data privacy and the necessity of documentation security, fully validating the architectural decision to integrate PostgreSQL `pgcrypto` for encrypted clinical records.

## Student Evaluation (UAT Results)

Preliminary User Acceptance Testing (UAT) was conducted with the student demographic to evaluate the system's usability, accessibility, and feature relevance. Based on the survey responses, the system received overwhelmingly positive feedback, with the vast majority of students rating their overall satisfaction at 4 or 5 out of 5.

- **Usability & Accessibility:** Students consistently highlighted the system's clean user interface and smooth navigation, giving exceptionally high ratings for the ease of logging in and the ability to access their records anytime and anywhere.
- **Highest Rated Features:** When asked which features were most useful, the student body overwhelmingly praised the **Patient Medical Dashboard**, the **Medical Certificate Issuance**, and the automated **BMI Calculation** tools. Students noted that "ordering" a medical certificate without waiting in physical queues was a massive operational improvement.
- **Constructive Feedback & Roadmap:** While technical issues were minimal, the most recurring constructive feedback was a request for a **"Dark Mode"** toggle and better responsiveness to ensure the application fits perfectly on narrower mobile screens.